

# Software Requirements Specification (SRS) – Online Used-Car Marketplace (Car24-like)

## Executive Summary

This SRS defines the requirements for a Car24-like online used-car marketplace. The system will support **buyers** (customers), **sellers** (private individuals or dealers), and **admin** users. Core features include searchable vehicle listings with **VIN decoding**, **vehicle history reports**, structured filters (make, model, price, etc.), and secure **checkout/payment** (including escrow). The platform will integrate **inspection services**, **financing/loan** and **insurance** options, **test-drive booking**, **real-time chat**, **reviews/ratings**, and **KYC/document uploads**. Administrators can approve listings, moderate content, and view analytics. Non-functional requirements cover high scalability, 99.9%+ availability, performance SLAs (e.g. sub-500ms search response), and robust security (HTTPS, OWASP best practices, PCI-DSS). Privacy will comply with GDPR/CCPA and KYC regulations. Third-party integrations (payment gateway, financing partners, vehicle registry APIs, SMS/Email services) are enumerated. The SRS includes detailed user stories, page descriptions, workflows (with Mermaid diagrams), a data model, API endpoint specifications, and a testing/deployment plan.

For example, Cars24 – a leading used-car platform – offers buying, selling, loans, and insurance services as part of its ecosystem [20†L108-L111] . Similarly, a marketplace must include features like VIN decoding (auto-filling vehicle specs via a third-party API) and vehicle-history integration (displaying accident/salvage records via a data API) [38†L175-L184] [38†L183-L189] . Additional “must-have” components are escrow payments, fraud-resistant messaging, saved searches with alerts, mandatory inspection reports, and tiered dealer/private seller flows [38†L175-L184] [38†L222-L228] [38†L230-L234] . Security and data protection are critical: e-commerce sites are mission-critical and must comply with regulations like GDPR (data controllers are responsible for compliance [34†L69-L74] ) and implement OWASP Top-10 mitigations [34†L79-L87] [34†L108-L112] .

## Overall Description

- **Product Perspective:** This platform is a full-stack web application (with a responsive design for desktop/mobile) enabling used-car transactions. It will have a web frontend, mobile-friendly UI, and backend services (likely microservices) on a cloud infrastructure. No specific technology stack or country is mandated; generic assumptions include a relational database (for listings, users, etc.), object storage for images, and standard web servers/APIs. The system will interoperate with external services (see *Third-Party Integrations* below) but is self-contained in core functionality.
- **Target Users:**

- **Buyers/Customers:** End-users looking to purchase a used car. They need to search/filter listings, view details (including history and inspection reports), chat with sellers, schedule test drives, and complete purchases (with payment/financing).
- **Sellers:** Individuals or authorized **dealers** who list vehicles for sale. They can create and manage listings (with photos, specs, VIN), respond to buyer inquiries/offers, and handle the sales process. Dealer accounts may have verification badges and higher listing limits 【38†L191-L199】 .
- **Admins:** Platform administrators who approve or reject listings, moderate content, manage user accounts, and view site-wide analytics (sales stats, traffic, etc.).
- **Support/Operations:** (Implied role) For tasks like customer support, manual KYC review, or vendor/finance partner liaisons.
- **Operating Environment:** Web browsers (Chrome, Safari, Edge, Firefox, etc.) on desktop/mobile; modern OS and browsers. The backend will run on a cloud provider (e.g. AWS/Azure/GCP) with containers or VMs, using HTTPS/TLS. The system should be mobile-responsive or have a companion mobile app. No legacy system integration is required beyond mentioned third-party APIs.
- **Assumptions:** Internet connectivity is available to all users. No specific legal jurisdiction is imposed, but standard e-commerce laws apply. Financial and personal data must be protected per global standards (e.g. GDPR, PCI-DSS). The product is greenfield; existing data is out of scope. If unspecified, adopt industry best practices.
- **Definitions & Abbreviations:** Key terms such as **VIN** (Vehicle Identification Number), **ESC** (escrow), **KYC** (Know Your Customer), **UI** (User Interface), **API** (Application Programming Interface), etc., are used as standard.

## Functional Requirements

The system's **functional requirements** (the “what”) cover all user-visible features and core operations 【25†L508-L517】 . Main features include:

- **User Accounts & Authentication:** Users (buyers/sellers/admins) can register (email/mobile with OTP) and log in. Seller onboarding requires identity verification (e.g. PAN/ID upload, dealer license). Role-based access controls separate buyer/seller/admin functions. Secure password storage and session management are required (e.g. JWT tokens, HTTPS-only cookies).
- **Vehicle Listing Management:**
  - **Create/Edit Listings:** Sellers enter or scan the car's VIN. The system auto-populates make, model, year, etc. via a VIN decoding API 【38†L175-L184】 . Sellers add photos (front, rear, interior, odometer, VIN plate, 360° view optional) and details (description, mileage, color, etc.). The platform enforces a minimum photo count and angle coverage to ensure transparency 【38†L212-L220】 【38†L214-L220】 . Sellers upload a signed *odometer disclosure* form and other legal documents if required.
  - **Listing Status:** New listings go into an approval queue. Admins review for compliance (fraud checks, completeness) before publishing. Listings can be “Active”, “Pending Approval”, “Sold”, or “Removed”. Upon sale, admin triggers any necessary process (e.g. escrow release).

- **Dealer vs. Private Flows:** Private individuals and dealers have separate flows. Dealers may get a verification badge and can bulk-upload via dealers' portal 【38†L191-L199】. Private sellers provide basic info and documents. Commission rates or fees may differ.
- **Search & Discovery:**
  - **Filters:** Buyers can search by keywords and filter by make, model, year, price range, body type, fuel type, transmission, location, mileage, etc. (structured filters). Saved filters and search alerts (email/push) notify buyers of new matching listings 【38†L198-L204】 【38†L230-L234】.
  - **Sorting:** Sort by price, newest, mileage, year, etc.
  - **Browse by Categories:** Pre-set categories (e.g. "Under ₹5L", "SUVs", "Electric") or featured cars on Home page.
  - **Search Index:** Full-text search on make/model and fast faceted queries. Ensure pagination and lazy-loading for large result sets.
- **Listing Detail Page:** For each car:
  - Large image gallery with zoom and swipe (mobile) features.
  - Vehicle specs (VIN-decoded data, year, mileage, engine, transmission, fuel, etc.).
  - Price (sticker price) and negotiation options (see Offers/Bidding).
  - *Vehicle History Report:* Integration with a data provider (e.g. carVertical, government database) shows accidents, salvage, insurance status 【38†L183-L189】.
  - *Inspection Report:* If a certified inspection was done, a downloadable report PDF is linked (uploaded by seller or inspection partner). Buyers can also book their own inspection.
  - Buttons for "**Make Offer**" (or "**Bid**"), "**Buy Now**", "**Schedule Test Drive**", and "**Contact Seller**" (opens chat).
  - Financing widget: Loan calculators (EMI) and *pre-qualification* link to partner lenders 【38†L260-L264】.
  - Insurance quote tool: Link to insurance partner with vehicle details pre-filled.
  - Seller info (name, badge, rating, location), and **reviews** of the seller (if any).
  - Social share and "Add to Wishlist".
- **Offers/Bidding and Purchase:**
  - **Fixed Price ("Buy Now"):** Buyer can instantly purchase at listed price.
  - **Offers/Negotiation:** Buyers can submit a lower *offer* via the platform. The seller sees pending offers and can accept, reject, or counter. Accepted offers proceed to purchase. Escrow may hold the payment until transaction completion 【38†L206-L214】.
  - **Checkout:** After buy/offer acceptance, buyer goes through checkout: confirm price, select payment method, and agree to terms. Optional: deposit an escrow amount via payment gateway (Stripe Connect or Razorpay). Payment covers price + platform fees (if any).
  - **Financing Integration:** Buyer can apply for a car loan. Pre-qualification checks (soft credit) integrate with partner APIs. Approved loans auto-insert finance terms into the order.

- **Order and Delivery:** Once paid (full or financed), the order is marked “Processing.” The seller arranges delivery or pickup. Ownership transfer (RC transfer) steps are initiated (system-generated e-forms).
- **Test-Drive Booking:** Buyers can request a test drive at a certified hub (e.g. Car24 hub) or at their home (with refundable fee). They pick a date/time, and the system notifies the seller/dealer. Calendar integration and reminders are sent.
- **Chat / Messaging:** In-app chat connects buyer and seller once an inquiry is made. Contact details (phone/email) remain hidden until after a secure connection (to prevent spam) [38†L222-L228] . The chat system supports images (e.g. extra photos) and has fraud-detection measures (keywords like “wire transfer”). A “Contact Support” chat with admins is also available.
- **Reviews & Ratings:** After a sale, buyers can rate the vehicle/seller (1–5 stars plus comments). Sellers can also rate buyers (for their punctuality, etc.). Reviews help build trust. Admins moderate reported reviews.
- **Payments:**
  - **Payment Gateway:** Integrate with a secure gateway (e.g. Razorpay, Stripe, or PayPal) for card/UPI/netbanking. Must support EMI/EMI-splitting if financing.
  - **Escrow:** Funds from buyer (or loan) are held in escrow and released to seller upon completion (e.g. after RC transfer). If deal falls through (inspection fail), refund flow is defined.
  - **Refunds/Cancellations:** Partial or full refunds handled per policy. Chargebacks handled per gateway rules.
  - **KYC & Document Upload:** To comply with financial regulations, KYC is required for buyers and sellers (via Aadhaar/PAN verification or equivalent services). Sellers upload RC (registration certificate), insurance papers, and identity proofs. These documents are stored securely (encrypted) and can be auto-verified via third-party APIs (e.g. eSign or KYC providers).
  - **Analytics & Reporting:** Admin dashboard provides metrics (visitors, leads, sales volume, inventory age, conversion rates, etc.). Sellers have a mini-dashboard: views per listing, inquiries, and sales earned. Data helps pricing tools (see below).
  - **Pricing Tools:** The platform offers dynamic pricing suggestions for sellers (based on recent sale data, depreciation tables, market trends). A simple **EMI/loan calculator** on each listing lets buyers estimate monthly payments.

All functional flows will be validated via user stories and use-case scenarios below. Core integrations are modular so as to treat third-party services as replaceable modules [25†L564-L571] .

## Page Navigation and UI

Key pages and their purposes are summarized in the table below:

| Page                      | Purpose / Key Elements                                                                                                                |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Home / Landing</b>     | Search bar (make/model), city selector, featured categories (New Listings, Deals, Popular Models), latest blog/promos.                |
| <b>Search / Results</b>   | Filter sidebar (make, model, price, year, etc.), sort options, and a list/grid of car cards (image, title, price, rating).            |
| <b>Listing Detail</b>     | Car image gallery, specs (from VIN), price, negotiation & buy buttons, loan calculator, seller info, and vehicle history.             |
| <b>Seller Dashboard</b>   | Overview (active listings, pending approvals, sold cars, earnings), "Create Listing" form, and listing management (edit, view stats). |
| <b>Buyer Dashboard</b>    | Saved searches & cars, inquiries (offers/test drives) status, order history, wallet/balance, and account settings.                    |
| <b>Admin Panel</b>        | New listing queue, reported content, user management (verify dealers, ban users), site analytics (traffic, sales).                    |
| <b>Authentication</b>     | Login/Signup pages (email/password or social/OAuth), password reset. Seller onboarding with additional KYC/document forms.            |
| <b>Checkout</b>           | Order summary (car + any fees), payment method selection, billing info, finalize payment.                                             |
| <b>Support &amp; Help</b> | FAQ, contact form, live chat support interface, and feedback form.                                                                    |
| <b>Legal/Policy</b>       | Static pages for Terms of Service, Privacy Policy, and disclosures.                                                                   |

Each page's UI will be mobile-responsive. Standard UX patterns are used: persistent top navigation (Home, Search, Sell, Login), breadcrumb on detail pages, and clear call-to-action buttons. Key UI flows (search → view car → test-drive/buy) ensure low friction for conversion.

## User Roles and Use Cases

### Buyers (Customers):

- *Use Case 1: Search & Find Car.* As a buyer, I can filter listings by criteria (e.g. "SUV, <2019, <₹10L") so I find suitable cars. (Acceptance: Relevant results appear in <1s.)
- *Use Case 2: View Details & History.* As a buyer, I can view a listing's detailed specs, photos, and history report, so I trust the information.
- *Use Case 3: Schedule Test Drive.* As a buyer, I can book a test-drive at a preferred time/location. The seller receives the request and confirms.
- *Use Case 4: Negotiate or Buy.* As a buyer, I can submit an offer lower than the listed price. If accepted, I proceed to checkout; or I can choose "Buy Now" to pay the full price immediately.
- *Use Case 5: Chat with Seller.* As a buyer, I can chat with the seller before deciding, asking questions about the car. My contact details remain hidden until we agree to transact.

- *Use Case 6: Checkout & Pay.* As a buyer, I can pay via saved card or other gateway, and see the payment confirmation immediately. If I finance the car, I can submit a loan application through the integrated partner.
- *Use Case 7: Leave Review.* After receiving the car, I can rate the seller/car with stars and comments.

#### **Sellers (Private/Dealer):**

- *Use Case 1: Create Listing.* As a seller, I can list my car by entering the VIN (auto-fill details) and uploading photos/docs. The listing saves as “Pending” until approved.
- *Use Case 2: Manage Listings.* As a seller, I can edit or remove my active listings. I can also mark a car as “Sold”.
- *Use Case 3: Receive Offers.* As a seller, I get notifications when buyers make offers or inquiries. I can accept/reject offers or counter with a new price.
- *Use Case 4: Confirm Sale.* Once a buyer pays (or deposit), I receive a sale confirmation and arrange the car’s handover. The system guides me through RC transfer steps.
- *Use Case 5: Dashboard Insights.* As a seller, I see how many views and inquiries my listings got, and monthly earnings summaries.

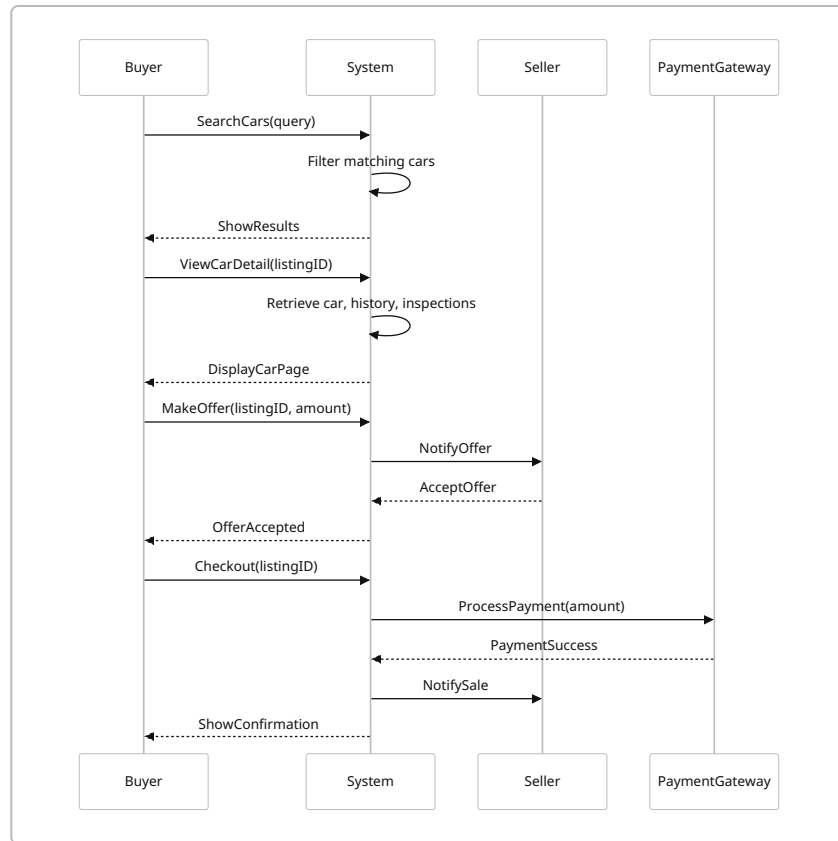
#### **Admins:**

- *Use Case 1: Listing Approval.* As an admin, I can review new listings in a queue. I approve compliant listings and reject those with issues (e.g. fraudulent, incomplete).
- *Use Case 2: User Management.* As an admin, I can verify dealer accounts, reset passwords, suspend malicious users, and manage roles.
- *Use Case 3: Content Moderation.* I can review flagged reviews/chats and delete spam or abusive content.
- *Use Case 4: Analytics & Reports.* I view dashboards (e.g. total sales by region, inventory aging) and generate periodic reports. I also monitor system health (uptime, error rates).

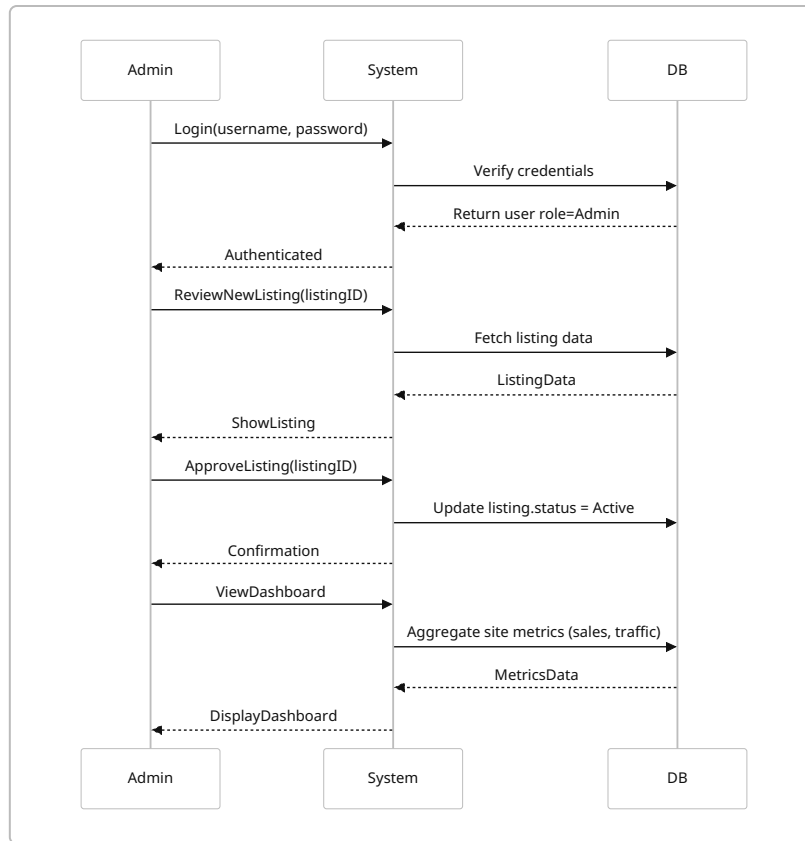
*(Additional roles such as “Finance Partner” or “Inspector” would interact via APIs or partner portals, but are outside this SRS’s user interface scope.)*

## **Workflows (Sequence Diagrams)**

Below is a simplified sequence diagram of a **buyer’s purchase flow**:



And an **admin's content moderation flow**:



These diagrams illustrate key interactions; actual implementation may involve more steps (e.g. session auth, error handling).

## Data Model (ER Tables)

Key entities and attributes are outlined below (ER diagram conceptually):

| Entity           | Key Attributes                                                                                                                                         |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User</b>      | <code>user_id</code> , name, email, password_hash, role (buyer/seller/dealer/admin), contact_info, KYC_status, created_at, updated_at                  |
| <b>Listing</b>   | <code>listing_id</code> , seller_id (→User), <code>vehicle_id</code> , title, description, price, status (pending/active/sold), created_at, expires_at |
| <b>Vehicle</b>   | <code>vehicle_id</code> , <code>vin</code> , make, model, year, trim, mileage, fuel_type, transmission, body_type, color                               |
| <b>Offer</b>     | <code>offer_id</code> , listing_id (→Listing), buyer_id (→User), amount, status (pending/accepted/rejected), created_at                                |
| <b>TestDrive</b> | <code>testdrive_id</code> , listing_id, buyer_id, scheduled_time, status (pending/confirmed/completed), location                                       |



| Entity             | Key Attributes                                                                                                                                                                                                     |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Order</b>       | <code>order_id</code> , <code>listing_id</code> , <code>buyer_id</code> , <code>seller_id</code> , <code>final_price</code> , <code>payment_status</code> , <code>order_date</code> , <code>delivery_status</code> |
| <b>Payment</b>     | <code>payment_id</code> , <code>order_id</code> , <code>amount</code> , <code>method</code> , <code>status</code> , <code>transaction_ref</code> , <code>created_at</code>                                         |
| <b>Review</b>      | <code>review_id</code> , <code>listing_id</code> , <code>user_id</code> (author), <code>rating</code> (1-5), <code>comment</code> , <code>created_at</code>                                                        |
| <b>ChatSession</b> | <code>chat_id</code> , <code>user1_id</code> , <code>user2_id</code> , <code>created_at</code>                                                                                                                     |
| <b>Message</b>     | <code>message_id</code> , <code>chat_id</code> , <code>sender_id</code> , <code>content</code> , <code>timestamp</code>                                                                                            |
| <b>LoanApp</b>     | <code>loan_id</code> , <code>listing_id</code> , <code>buyer_id</code> , <code>lender_id</code> , <code>amount</code> , <code>status</code>                                                                        |
| <b>DocUpload</b>   | <code>doc_id</code> , <code>user_id/listing_id</code> , <code>doc_type</code> (e.g. RC, insurance, ID), <code>file_url</code> , <code>verified_flag</code>                                                         |
| <b>Category</b>    | (lookup) e.g. makes, models, cities for filters.                                                                                                                                                                   |

Each entity's details (data types, constraints) would be finalized during design. Notably, **VIN** and **vehicle specs** come from external APIs [38†L175-L184], so the `Vehicle` record is populated via service calls. A normalized schema ensures we can query, for example, all active listings for a given model or aggregate sales by location. Sensitive fields (user credentials, docs) are encrypted and access-logged.

## API Endpoints

All functionality is exposed via RESTful JSON APIs. Below is a sample table of key endpoints (examples are illustrative):

| Endpoint                      | Method | Description                              | Request / Response (JSON)                                                                                                                                                                                                                 |
|-------------------------------|--------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/api/auth/signup</code> | POST   | Register user (buyer or seller)          | <b>Request:</b> <code>{ "email": "alice@example.com", "password": "Secret123", "role": "buyer" }</code><br><b>Response:</b> <code>201 Created { "user_id": 1001, "message": "Signup successful" }</code>                                  |
| <code>/api/auth/login</code>  | POST   | Login (returns auth token)               | <b>Request:</b> <code>{ "email": "alice@example.com", "password": "Secret123" }</code><br><b>Response:</b> <code>200 OK { "token": "&lt;JWT_TOKEN&gt;" }</code>                                                                           |
| <code>/api/listings</code>    | GET    | Search/list cars (with filters in query) | <b>Request:</b> <code>GET /api/listings?make=Honda&amp;year=2019&amp;max_price=500000</code><br><b>Response:</b> <code>200 OK [ { "listing_id": 200, "title": "Honda City 2019", "price": 480000, "thumbnail": "...", ... }, ... ]</code> |

| Endpoint                  | Method | Description                        | Request / Response (JSON)                                                                                                                                                                                                   |
|---------------------------|--------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /api/listings/{id}        | GET    | Get details of one listing         | <b>Response:</b><br>200 OK { "listing_id":200, "title":"Honda City 2019", "make":"Honda", "model":"City", "year":2019, "mileage":35000, "price":480000, "status":"active", ... }                                            |
| /api/listings             | POST   | Create a new listing (seller only) | <b>Request:</b> { "seller_id": 300, "vin":"1HGCM82633A004352", "title":"...", "price":450000, "images":[ "...base64..." ], "description":"", ... }<br><b>Response:</b> 201 Created { "listing_id":201, "status":"pending" } |
| /api/listings/{id}        | PUT    | Update a listing (seller only)     | <b>Request:</b> { "price":440000, "description":"Reduced price!" }<br><b>Response:</b> 200 OK { "message":"Updated" }                                                                                                       |
| /api/listings/{id}/offers | POST   | Make an offer on listing (buyer)   | <b>Request:</b> { "buyer_id": 101, "offer_price":420000 }<br><b>Response:</b> 201 Created { "offer_id":5001, "status":"pending" }                                                                                           |
| /api/users/{id}           | GET    | Get user profile (protected)       | <b>Response:</b><br>200 OK { "user_id":1001, "name":"Alice", "role":"buyer", ... }                                                                                                                                          |
| /api/testdrives           | POST   | Schedule a test drive (buyer)      | <b>Request:</b> { "buyer_id":101, "listing_id":200, "datetime":"2026-09-15T10:00Z", "location":"Hub Delhi" }<br><b>Response:</b> 201 Created { "testdrive_id":7001 }                                                        |
| /api/payments/checkout    | POST   | Process payment (checkout)         | <b>Request:</b> { "order_id": 9001, "payment_token":"<gatewayToken>" }<br><b>Response:</b> 200 OK { "payment_id": 5501, "status":"success" }                                                                                |
| /api/chat                 | POST   | Send chat message                  | <b>Request:</b> { "chat_id": 8001, "sender_id":1001, "message":"Is the car still available?" }<br><b>Response:</b> 200 OK { "message_id":9001, "timestamp": "..."}<br>                                                      |
| /api/reviews              | POST   | Submit a review                    | <b>Request:</b> { "listing_id":200, "user_id":101, "rating":5, "comment":"Great car!" }<br><b>Response:</b> 201 Created { "review_id":6001 }                                                                                |

| Endpoint                                | Method | Description                                    | Request / Response (JSON)                                                          |
|-----------------------------------------|--------|------------------------------------------------|------------------------------------------------------------------------------------|
| /api/<br>admin/<br>listings/<br>pending | GET    | List<br>pending<br>listings<br>(admin<br>only) | <b>Response:</b><br>200 OK [ { "listing_id":201, "seller_id":<br>300, ... }, ... ] |

*Note:* All endpoints require authentication (except signup/login). JSON Web Tokens (JWT) or similar are used for session auth. Error handling returns standard HTTP codes (4xx for client errors, 5xx for server errors) with descriptive messages.

## UI Wireframes (Descriptions)

While detailed mockups are outside this SRS, key UI wireframe descriptions include:

- **Home Page:** Hero section with search (prominent filters), quick links (e.g. “Sell Your Car”), and banners for promotions (e.g. “Zero down payment on select cars”). Below, featured categories (by price or body type) and recent listings as cards (photo + basics).
- **Search Results:** Two-column layout: left sidebar of filters (accordion style), main area listing car cards. Each card shows image carousel, title (make/model), key specs (year, mileage), price, and a “View Details” button.
- **Listing Detail:** Top has a large carousel of images. Left panel shows price, seller info (with rating), action buttons (Contact, Offer, Buy, Test Drive). Below, tabs or sections: *Vehicle Details* (table of specs), *Description*, *History & Inspection Reports* (downloadable PDFs), *Financing Offers*. User reviews appear at bottom.
- **Seller Dashboard:** Tiles summarizing “Active Listings”, “Offers Received”, “Pending Approval”. A table of the seller’s listings with quick actions (Edit, View, Mark Sold). Forms for adding/editing a listing appear in a modal or dedicated page, with VIN lookup and photo uploader.
- **Buyer Dashboard:** Sections for “Saved Cars”, “My Inquiries” (offers/test drives), and “Orders”. Each order shows status (e.g. Payment Pending, Shipped). A profile area for personal info and KYC documents.
- **Admin Panel:** Sidebar menu (Listings, Users, Reports, Settings). The “Pending Listings” page lists new submissions with thumbnail, seller name, and approve/reject buttons. Dashboard shows charts (e.g. monthly sales, active users) and tables of recent activity.

Visual consistency (colors, typography) and intuitive flow are priorities. Interactive elements (buttons, inputs) follow accessibility guidelines (ARIA labels, keyboard support).

## Non-Functional Requirements

- **Performance:** The site must have fast responses. **Search** queries should return results in under *500ms* on average. **Page loads** (desktop) should be under *3 seconds* (even on modest mobile networks). SLAs might specify 95% of requests must meet these targets [25†L437-L444] . CDN (for images) and server-side caching (Redis or Memcached) will be used to speed repeated accesses.
- **Scalability:** Design for horizontal scaling. The architecture should auto-scale to handle peak traffic (e.g. 10K concurrent users) without degradation. A multi-tier cloud setup (with load-balancing and stateless app servers) is recommended. Scalability targets (traffic volume, inventory size, internationalization) should be documented [25†L439-L444] .
- **Availability:** Aim for **99.9% uptime** or better. The system will deploy across multiple availability zones (AZs) for failover. Use managed databases with replica failover, health-checking load balancers, and automated backups. **DDoS protection** (e.g. AWS Shield) and WAF rules protect critical endpoints.
- **Security:** Security is mission-critical for e-commerce [34†L79-L87] . All data in transit must use HTTPS/TLS. Data at rest (e.g. user PII, documents) is encrypted. Authentication must prevent attacks (strong password policy, bcrypt hashing, JWT expiration). Input is validated/sanitized to prevent SQL injection or XSS. The system must mitigate OWASP Top-10 risks (e.g. CSRF tokens, rate limiting, secure cookies). Payment flows must be PCI-DSS compliant. Regular vulnerability scanning and third-party security audits are required. As Kinsta notes, poor security can ruin reputation: “a security flaw can put a company’s life at risk” [34†L79-L87] .
- **Data Privacy & Compliance:** Personal data handling must comply with **GDPR** (for EU users) and similar laws. Per GDPR, the company is the data controller responsible for compliance [34†L69-L74] . Users have rights (data access, deletion requests). The privacy policy must disclose data uses. KYC data (government IDs) must be handled per legal guidelines (e.g. stored only as long as needed, encrypted). If operating in India or US, local laws (e.g. IT Act, CCPA) apply similarly.
- **Maintainability:** Codebase should follow modular design (e.g. microservices or well-structured monolith), with clear APIs. Automatic testing and CI/CD pipelines ensure reliability. Configurable settings (feature flags for new modules) allow incremental rollout. The architecture should minimize single points of failure and use Infrastructure-as-Code (IaC) for repeatable deployments.
- **Usability:** The UI/UX should be intuitive and minimal-friction. The design must work on common browsers/devices (including adaptive/responsive mobile layout). Accessibility standards (WCAG 2.1 AA) ensure use by people with disabilities. Multi-lingual support may be considered (but not mandatory unless expanded globally).
- **Logging & Monitoring:** All critical actions (logins, listing creations, payments) are logged with timestamps and user IDs. Logs feed into a centralized system (ELK stack or CloudWatch) for analysis. Key metrics (CPU, response times, error rates) are monitored with alerts. GDPR requires logging access to personal data (audit trail). Monitoring also includes synthetic transactions (regular health-check of login, search, checkout flows).

## Data Privacy / KYC Considerations

- **GDPR/CCPA:** User consent must be obtained for email notifications and tracking cookies. Provide “Download My Data” and “Delete Account” options. Any data breach must be reported within regulatory deadlines.
- **KYC/AML:** For buyers arranging financing or large transactions, perform KYC as required by financial regulators (e.g. verify identity documents via APIs like Onfido or Karza). Sellers must be identified (PAN/Aadhaar for individuals; dealership license for dealers).
- **Secure Document Handling:** Sensitive docs (Aadhaar, RC) are stored encrypted, accessible only to authorized services/admins. OCR and data extraction (with user consent) expedite verification. After sale, retention of documents follows local law (e.g. financial records kept 3–5 years, then deleted).

Compliance steps (privacy policy, user agreements) are outside this SRS, but technical design must enable them.

## Third-Party Integrations

The platform will integrate several external services via APIs **【25†L564-L571】** :

- **Payment Gateways:** E.g. Razorpay (India), Stripe, or PayPal. Supports UPI, cards, netbanking. Must securely handle callbacks/webhooks.
- **Bank/Financing APIs:** Partner with banks or NBFCs for auto loans (e.g. HDFC Bank’s API, Bajaj Finserv, or a loan aggregator). The site can pre-fill applications and handle loan disbursement/EMI details.
- **Insurance API:** Connect to insurers (e.g. ICICI Lombard, TATA AIG) for instant premium quotes or to renew insurance. Provide quick-bind or referrals.
- **Vehicle Registry (RTO) APIs:** Use government (e.g. India’s Parivahan) or services like Attestr/Surepass to fetch RC details from registration numbers (vehicle age, owner, registration status). Provide challan (traffic fine) checks and FASTag balance lookups as in Car24’s *Vehicle Info* acquisition **【20†L46-L54】** .
- **VIN/Vehicle Data:** API for decoding VIN into specs (e.g. VINData, CarQuery API).
- **Vehicle History/Data:** Carfax/AutoCheck (for US markets) or carVertical (international) to fetch accident/title history.
- **Inspection Partners:** Integration with a service like Tchek or local car clinics: buyers can book an inspection through the platform, which sends details to a partner; results are returned/uploaded as reports. Sellers might also use pre-purchase inspections (upload PDF of independent inspection).
- **Messaging/SMS/Email:** Twilio or similar for SMS OTPs and notifications; SendGrid or Amazon SES for emails. Slack/MS Teams integration for admin alerts (optional).
- **Maps/Location:** Google Maps or Mapbox for geolocation (dealer addresses, test-drive mapping).
- **Analytics:** (Secondary) Google Analytics / Mixpanel for user behavior, plus BI tools for admin analytics.
- **Other:** Social login (Google/Facebook) if desired. E-signature API (e.g. DocuSign or India’s eMudhra) for digital forms (e.g. agreement signatures).

Each integration is encapsulated so swapping providers is possible. The SRS assumes these services exist; detailed provider selection is a project decision. All third-party data flows must use secure, authenticated APIs. As Elogic advises, list all required integrations early to avoid scope gaps **【25†L564-L571】** .

## Testing Plan

A thorough testing strategy includes:

- **Unit Testing:** Developers write unit tests for all backend services and UI components (target  $\geq 80\%$  coverage). E.g. test search algorithms, price calculations, API controllers.
- **Integration Testing:** Test end-to-end flows between components and with third-party mocks (e.g. simulate payment gateway response, VIN API, etc.). Use a test DB instance.
- **User Acceptance Testing (UAT):** Involve real users/stakeholders to run through key scenarios. Validate that all user stories work as expected (e.g. listing creation, checkout flow). Feedback cycles refine requirements.
- **Performance Testing:** Use tools like JMeter or k6 to simulate load on critical paths (search, checkout). Ensure the system meets defined benchmarks (e.g. search  $< 0.5s$  under load). Scalability tests (auto-scaling triggers) and spike tests (flash sale simulation) are included.
- **Security Testing:** Perform OWASP Top-10 checks (SQLi, XSS) with static analysis and penetration testing tools. Ensure no sensitive data leaks in logs. PCI DSS scan for payment flows.
- **Regression Testing:** Before each release, run automated regression suite plus smoke tests on major features.
- **Monitoring & Error Tracking:** In production-like staging, set up automated monitoring (New Relic/Datadog) and error tracking (Sentry). Alert on any exceptions or slow endpoints.

Acceptance criteria: All critical defects fixed, and the application passes a “definition of done” sign-off by stakeholders. Performance metrics (TPS, latency) must meet SLAs. Security audits must reveal no high-severity issues.

## Deployment Architecture

The system will be deployed on cloud infrastructure for reliability and scalability. Below is an example AWS architecture (multi-AZ):

【53†embed\_image】 *Figure: Sample cloud deployment architecture using CDN, load balancers, auto-scaling app servers, cache layers, and multi-AZ database (AWS example).*

In this architecture, edge traffic routes through a **Content Delivery Network (CDN)** and **Web Application Firewall (WAF)** 【52†L20-L28】 for DDoS protection. An **API Gateway** or **Load Balancer** distributes requests to **auto-scaled** application servers (e.g. in Kubernetes or EC2). A **relational DB** (e.g. AWS Aurora, MySQL/Postgres) runs in primary-replica mode across AZs. In-memory caches (Redis/Memcached) speed up sessions and frequent queries. Static assets (images, docs) are stored in object storage (S3) and served via CDN. Infrastructure is managed via IaC (Terraform/CloudFormation). Backup and failover strategies ensure data safety (e.g. RDS automated snapshots, cross-AZ replicas). Monitoring services (CloudWatch, Grafana) collect logs and metrics. Overall, this meets requirements for high availability, auto-scaling, and performance 【52†L20-L28】 .

## Monitoring and Logging

The platform will incorporate real-time monitoring and logging:

- **Application Monitoring:** Use Prometheus/Grafana or a cloud APM to track service metrics (CPU, memory, request latency, error rates).
- **Log Management:** Centralize logs (ELK stack or cloud equivalent) to index application logs, access logs, and security logs. Protect logs from tampering and ensure retention policy compliance.
- **Alerts:** Configure alerts for critical events (e.g. payment failures, high error rate, downtime). Use email/SMS alerts or integrate with incident management (PagerDuty).
- **Audit Trail:** Key actions (admin changes, price updates, KYC status changes) are logged for auditability. Any data access (especially PII) is logged per compliance needs.

These NFRs ensure we maintain the system's health and can diagnose issues promptly.

## Acceptance Criteria

- **Functionality:** Every functional requirement is implemented and demonstrated. For each feature (search, listing, checkout, etc.), acceptance tests pass (as defined in the test plan).
- **Performance:** The site meets defined SLAs (e.g. 95th percentile page load <3s, API response <0.5s under normal load) [25†L437-L444] .
- **Reliability:** 99.9% uptime in production (excluding scheduled maintenance). All failover mechanisms have been tested.
- **Security:** No critical vulnerabilities remain (verified by audit). Data encryption and secure protocols are in place. PCI and GDPR compliance requirements are satisfied.
- **Usability:** UI/UX passes basic usability tests; no major navigation issues. Mobile responsiveness works on target devices.
- **Regulatory:** KYC processes cover mandatory fields; privacy policy is enforced (e.g. user consent mechanisms).
- **Stakeholder Sign-off:** Product owner and business stakeholders have formally approved the system.

Each requirement is **verifiable** (through tests, demonstrations, or inspections) and tied to a user story or business goal. Per IEEE SRS practices, no requirement is ambiguous or unspecifiable [25†L508-L517] .

---

**Sources:** Core requirements and best practices were informed by examples from existing marketplaces (e.g. Cars24) and e-commerce SRS guidelines [20†L108-L111] [38†L175-L184] [25†L508-L517] [34†L69-L74] . The specification above compiles industry standards into a coherent SRS for development guidance.

---